Programming fundamentals

Credit Hours: 3-1

Course Outline

Introduction to Programming Concepts, Algorithms and Flowcharts, Basics of Computer Systems and Programming Languages, Data Types and Variables, Input and Output Operations, Operators and Expressions, Control Structures (Selection and Iteration), Functions and Parameter Passing, Arrays (One-dimensional and Multi-dimensional), Strings and String Handling, Pointers and Memory Management, Structures and Unions, File Handling, Introduction to Object-Oriented Programming Concepts, Debugging and Error Handling, Basic Problem Solving and Program Design Techniques

16-Week Course Plan

Week 1

Week 1: Introduction to Programming Concepts

3-1

Lecture 1: Introduction to Programming

Learning Objectives

• Understand the basic concepts of programming

• Learn the importance of programming in today's world

• Identify the different types of programming languages

Concepts

Programming is the process of designing, writing, testing, debugging, and maintaining the source code of computer programs. Programming languages are used to write programs that can be executed by computers. There are two main types of programming languages: high-level languages (e.g., Python, Java) and low-level languages (e.g., Assembly). Programming is essential in today's world as it powers most of the technological advancements.

Lecture 2: Basics of Computer Systems

Learning Objectives

• Understand the basic components of a computer system

• Learn the role of hardware and software in a computer system

• Identify the different types of computer systems

Concepts

A computer system consists of hardware and software components. Hardware components include the central processing unit (CPU), memory (RAM), storage devices (hard drive, SSD), and input/output devices (keyboard, mouse, monitor). Software components include the operating system, programming languages, and applications. Computer systems can be classified into three types: single-user systems, multi-user systems, and embedded systems.

Lecture 3: Basics of Programming Languages

Learning Objectives

• Understand the basic concepts of programming languages

• Learn the different types of programming languages

• Identify the characteristics of high-level and low-level languages

Concepts

Programming languages are used to write programs that can be executed by computers. There are two main types of programming languages: high-level languages (e.g., Python, Java) and low-level languages (e.g., Assembly). High-level languages are easier to learn and use, while low-level languages provide direct access to hardware resources.

Lab 1: Introduction to Programming Environments

Learning Objectives

• Learn to set up a programming environment

• Understand the basics of a text editor or IDE

• Write and run a simple program

Concepts

This lab introduces students to the basics of a programming environment, including setting up a text editor or IDE, writing and running a simple program. Students will learn to write and execute their first program, which will lay the foundation for further programming concepts.

Week 2

Week 2: Basics of Computer Systems and Programming Languages (3-1)

Lecture 1: Introduction to Computer Systems

Learning Objectives

* Understand the basic components of a computer system

* Identify the roles of hardware and software in a computer system

Concepts

A computer system consists of hardware and software components. Hardware includes the CPU, memory, storage devices, and input/output devices. Software includes the operating system, application software, and programming languages. The CPU executes instructions, memory stores data and programs, storage devices hold data and programs, and input/output devices interact with the user.

Lecture 2: Programming Languages

Learning Objectives

* Understand the characteristics of high-level programming languages

* Identify the importance of programming languages in software development

Concepts

High-level programming languages are abstract and far removed from machine language. They provide features such as variables, data types, control structures, and functions. Examples of high-level programming languages include Java, Python, and C++. Programming languages play a crucial role in software development as they enable developers to write efficient, reliable, and

maintainable code.

Lecture 3: Language Families

Learning Objectives

* Understand the different language families and their characteristics

* Identify the relationships between language families

Concepts

Programming languages can be classified into several language families, including procedural, object-oriented, functional, and logic programming languages. Procedural languages, such as C and Pascal, use procedures and functions to organize code. Object-oriented languages, such as Java and C++, use objects and classes to organize code. Functional languages, such as Lisp and Scheme, use functions as first-class citizens. Logic programming languages, such as Prolog, use logical statements to solve problems.

Lab 1: Programming Language Selection

Learning Objectives

* Understand the importance of selecting the right programming language for a project

* Practice selecting a programming language for a given project

Concepts

In this lab, students will practice selecting a programming language for a given project. They will consider factors such as the project's requirements, the language's features, and the language's popularity. Students will also research and evaluate different programming languages to determine the best fit for the project.

Week 3

Week 3: Basics of Computer Systems and Programming Languages (3-1)

Lecture 1: Introduction to Computer Systems

Learning Objectives

* Understand the basic components of a computer system

* Identify the roles of hardware and software in a computer system

Concepts

A computer system consists of hardware and software components. Hardware includes the central processing unit (CPU), memory, input/output devices, and storage devices. Software includes the operating system, applications, and programming languages. The CPU executes instructions, memory stores data, input/output devices interact with the user, and storage devices hold data and programs.

Lecture 2: Introduction to Programming Languages

Learning Objectives

* Define a programming language

* Identify the characteristics of a programming language

* Explain the importance of programming languages

Concepts

A programming language is a set of instructions that a computer can execute. Characteristics of a programming language include syntax, semantics, and pragmatics. Syntax refers to the rules that govern the structure of a program, semantics refer to the meaning of a program, and pragmatics refer to the efficiency and effectiveness of a program. Programming languages are essential for communicating with computers and automating tasks.

Lecture 3: Language Paradigms

Learning Objectives

* Define language paradigms

* Identify the main language paradigms

* Explain the characteristics of each paradigm

Concepts

Language paradigms refer to the programming style or philosophy of a language. The main language paradigms include imperative, object-oriented, functional, and logic programming. Imperative programming focuses on actions and steps, object-oriented programming emphasizes objects and classes, functional programming uses pure functions and recursion, and logic programming uses logical statements and inference.

Lab 1: Exploring Programming Languages

Learning Objectives

* Install and configure a programming environment

* Write and execute a simple program in a chosen language

* Compare the syntax and semantics of different languages

Concepts

In this lab, students will install and configure a programming environment, write and execute a simple program in a chosen language, and compare the syntax and semantics of different languages. This lab aims to introduce students to the basics of programming languages and prepare them for more advanced topics.

Week 4

Week 4: Control Structures (Iteration)

Lecture Title: Introduction to Loops

Learning Objectives

• Understand the concept of iteration in programming

• Learn the syntax and application of for and while loops

• Understand the use of break and continue statements

Concepts

Loops in programming allow code to execute repeatedly for a specified number of times or until a condition is met. The two primary types of loops are for loops and while loops. For loops are used when the number of iterations is known in advance, whereas while loops continue to execute as long as a condition is true. The break statement is used to exit a loop prematurely, while the continue statement skips the current iteration and moves on to the next one. Understanding the control flow within loops is essential for writing efficient and effective programs.

Lecture Title: Understanding Loop Control

Learning Objectives

• Understand the use of loop control statements (break and continue)

• Learn how to write conditional loops with if statements inside

• Understand the importance of loop termination conditions

Concepts

Loop control statements are crucial for managing the flow of a program within loops. The break statement is used to terminate the loop immediately, while the continue statement skips the current iteration and continues with the next one. Conditional loops can be created by combining if statements with loops, allowing for more complex decision-making within the loop. The termination condition of a loop must be precise to avoid infinite loops or incorrect execution.

Lecture Title: Loop Examples and Best Practices

Learning Objectives

• Understand how to apply loops in real-world programming scenarios

• Learn best practices for writing efficient loops

• Understand the importance of testing loops thoroughly

Concepts

Loops are fundamental to programming and are used extensively in various applications. Understanding how to apply loops effectively requires a combination of theoretical knowledge and practical experience. Best practices for writing loops include keeping loops concise, avoiding unnecessary iterations, and thoroughly testing loop logic. Effective use of loops can significantly improve program performance and reduce development time.

Lab: Looping Exercises

Lab Hours: 1

In this lab, students will practice writing loops with different control statements, loop types, and applications. They will also learn to identify and correct common loop-related errors.

Week 5

Week 5: Arrays (One-dimensional and Multi-dimensional)

3-1

Lecture 1: Introduction to Arrays

Learning Objectives

• Define an array and its types

• Declare and initialize one-dimensional arrays

• Understand array indexing and bounds checking

Concepts

An array is a collection of elements of the same data type stored in contiguous memory locations. Arrays can be one-dimensional (1D) or multi-dimensional (2D, 3D, etc.). In a 1D array, elements are stored in a single row or column. To declare a 1D array, we use the data type followed by the array name and the number of elements in square brackets. For example, int arr[5]; declares an integer array arr with 5 elements.

Lecture 2: Multi-dimensional Arrays

Learning Objectives

• Understand the concept of multi-dimensional arrays

• Declare and initialize multi-dimensional arrays

• Understand how to access elements in a multi-dimensional array

Concepts

A multi-dimensional array is an array of arrays. In a 2D array, each element is an array itself. To declare a 2D array, we use the data type followed by the array name and the number of rows and columns in square brackets. For example, int arr[3][4]; declares a 2D integer array arr with 3 rows and 4 columns.

Lecture 3: Array Operations and Applications

Learning Objectives

• Understand array operations such as traversal and searching

• Learn how to use arrays in real-world applications

• Understand the advantages and disadvantages of using arrays

Concepts

Arrays are useful for storing and manipulating large amounts of data. They can be used to implement algorithms such as searching and sorting. Arrays are also used in various real-world applications such as data analysis, scientific computing, and game development.

Lab 1: Working with Arrays

Lab Objective: Write a C program to perform various operations on a 2D array, such as printing, searching, and sorting.

Lab Instructions:

1. Declare a 2D array and initialize it with values.

2. Write a function to print the array.

3. Write a function to search for a specific element in the array.

4. Write a function to sort the array in ascending order.

5. Test the functions using sample input and output.

Week 6

Week 6: Functions and Parameter Passing

3-1

Functions and Parameter Passing

Learning Objectives:

• Understand the concept of functions and their purpose in programming.

• Learn how to declare and call functions.

• Understand the different types of parameter passing: call-by-value, call-by-reference, and call-by-result.

Concepts:

Functions are reusable blocks of code that perform a specific task. They take arguments, execute a set of statements, and return values. Functions are essential in modular programming, making code more organized, maintainable, and efficient.

Types of Parameter Passing:

* **Call-by-Value:** The function receives a copy of the original value.

* **Call-by-Reference:** The function receives the memory address of the original value.

* **Call-by-Result:** The function receives the result of an expression.

Declaring and Calling Functions:

* Function declaration: `return-type function-name(parameters)`

* Function call: `function-name(arguments)`

Example:

```c

int add(int x, int y) {
```

```
return x + y;

}

int main() {

int result = add(5, 7);

printf("%d\n", result);

return 0;

}
```

Lab:

1. Declare a function that takes two integers as arguments and returns their sum.

2. Call the function with two integer arguments and print the result.

3. Experiment with different types of parameter passing.

Week 7

Week 7: File Handling and Persistence

3-1

Lecture 1: Overview of File Handling

Learning Objectives

* Understand the concept of file handling and persistence

* Learn to create and manage files in a program

Concepts

File handling is the process of reading and writing data to a file. Files are used to store data persistently, allowing programs to access and manipulate data even after the program has terminated. In this lecture, we will cover the basics of file handling, including file creation, reading, and writing.

Lecture 2: File Operations

Learning Objectives

* Learn to perform various file operations, such as opening and closing files, reading and writing data

* Understand the importance of file modes and permissions

Concepts

File operations are the actions performed on files, such as opening, closing, reading, and writing data. We will cover the various file operations, including file modes (text mode, binary mode) and

permissions (read, write, execute).

Lecture 3: File Input/Output Operations

Learning Objectives

* Learn to perform file input/output operations using standard input/output functions

* Understand the differences between file input/output and console input/output

Concepts

File input/output operations are performed using standard input/output functions, such as `fopen()`, `fread()`, and `fwrite()`. We will cover the differences between file input/output and console input/output, and how to use file input/output functions in a program.

Lab 1: File Handling Exercises

* Create a program that creates a file and writes data to it

* Create a program that reads data from a file and displays it on the console

* Experiment with different file modes and permissions

Week 8

Week 8: Arrays (One-dimensional and Multi-dimensional)

Lecture Title: Arrays in Programming

Learning Objectives:

• Define an array and its characteristics

• Declare and initialize one-dimensional and multi-dimensional arrays in a programming language

• Access and manipulate array elements

• Understand array indexing and bounds checking

Concepts:

An array is a collection of elements of the same data type stored in contiguous memory locations. Arrays are used to store and manipulate collections of data.

One-dimensional arrays:

• A one-dimensional array is a sequence of elements of the same data type stored in contiguous memory locations.

• It is declared using square brackets `[]` after the variable name, e.g., `int scores[]`.

• Elements are accessed using the index, e.g., `scores[0]`.

Multi-dimensional arrays:

• A multi-dimensional array is an array of arrays.

• It is declared using multiple pairs of square brackets `[][]`, e.g., `int scores[][]`.

• Elements are accessed using two or more indices, e.g., `scores[0][0]`.

Example:

```c
int scores[3][3] = {
{10, 20, 30},
{40, 50, 60},
{70, 80, 90}
};
```

Lab Exercise:

Create a program to demonstrate the usage of one-dimensional and multi-dimensional arrays.

Week 9

Week 9: Introduction to Object-Oriented Programming Concepts (3-0)

Lecture Title: Introduction to Classes and Objects

Learning Objectives

• Define classes and objects in OOP.

• Identify the concepts of inheritance, polymorphism, and encapsulation.

• Explain the difference between abstract and concrete classes.

Concepts

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. A class is a blueprint or a template that defines the properties and behaviors of an object. An object, on the other hand, is an instance of a class.

In OOP, classes can inherit properties and behaviors from parent classes, allowing for code reuse and a more modular design. Polymorphism enables objects of different classes to be treated as objects of a common superclass. Encapsulation helps hide the implementation details of an object from the outside world, promoting data hiding.

Abstract classes are classes that cannot be instantiated and are used to provide a common interface for subclasses. Concrete classes, however, are classes that can be instantiated and provide a specific implementation.

Object-Oriented Programming provides a robust and scalable approach to software development, making it a fundamental concept in modern programming.

Week 10

Week 10: 2-1

Lecture 1: Introduction to Object-Oriented Programming (OOP) Concepts

Encapsulation in OOP

Learning Objectives

• Understand the concept of encapsulation in OOP.

• Explain the benefits of encapsulation.

• Provide examples of encapsulation.

Concepts

Encapsulation is a fundamental concept in Object-Oriented Programming (OOP) that binds together the data and the methods that manipulate that data. It hides the internal details of an object from the outside world and only reveals the necessary information through a well-defined interface. Encapsulation provides several benefits, including:

• Improved data security and integrity

• Reduced complexity

• Easier maintenance and extension

• Improved reusability

Lecture 2: Inheritance in OOP

Inheritance in OOP

Learning Objectives

• Understand the concept of inheritance in OOP.

• Explain the benefits and types of inheritance.

• Provide examples of inheritance.

Concepts

Inheritance is a mechanism in OOP that allows one class to inherit the properties and behavior of another class. The inherited class is called the subclass or derived class, and the class from which it inherits is called the superclass or base class. Inheritance provides several benefits, including:

• Code reuse

• Reduced complexity

• Improved modularity

• Easier maintenance and extension

• Types of inheritance: single inheritance, multiple inheritance, multilevel inheritance

Lecture 3: Polymorphism in OOP

Polymorphism in OOP

Learning Objectives

• Understand the concept of polymorphism in OOP.

• Explain the benefits and types of polymorphism.

• Provide examples of polymorphism.

Concepts

Polymorphism is a mechanism in OOP that allows objects of different classes to be treated as objects of a common superclass. This is achieved through method overriding or method overloading. Polymorphism provides several benefits, including:

• Improved flexibility

• Reduced complexity

• Improved modularity

• Easier maintenance and extension

• Types of polymorphism: method overriding, method overloading

Lab 1: OOP Implementation

Lab: Implementing OOP Concepts

Learning Objectives

• Implement encapsulation, inheritance, and polymorphism in a programming language.

• Analyze the benefits of OOP concepts.

• Provide examples of OOP implementation.

Concepts

Students will implement OOP concepts in a programming language, such as Java or C++. They will create classes, objects, and methods to demonstrate encapsulation, inheritance, and polymorphism. The lab will help students understand the practical applications of OOP concepts and their benefits.

Week 11

Week 11: Pointers and Memory Management (3-1)

Lecture 1: Introduction to Pointers

Learning Objectives

• Understand the concept of pointers in programming.

• Learn how to declare and initialize pointers.

• Understand the difference between pointers and variables.

Concepts

A pointer is a variable that stores the memory address of another variable. Pointers are used to indirectly access and manipulate variables. In most programming languages, pointers are declared using the asterisk symbol (*). For example, `int *ptr;` declares a pointer to an integer variable. Pointers can be initialized using the address-of operator (&), for example, `int x = 10; int *ptr = &x;`.

Lecture 2: Pointer Arithmetic and Operations

Learning Objectives

• Understand how to perform arithmetic operations on pointers.

• Learn how to use pointer arithmetic to access elements in arrays.

• Understand the concept of pointer operations such as increment and decrement.

Concepts

Pointer arithmetic involves performing operations on pointers to access different memory locations. For example, `ptr++` increments the pointer to point to the next element in an array. Pointers can also be decremented using `ptr--`. Pointer operations can be used to access elements in arrays, for example, `ptr = arr + i;` points to the `i-th` element in the array `arr`.

Lecture 3: Memory Management with Pointers

Learning Objectives

• Understand how to allocate and deallocate memory using pointers.

• Learn how to use dynamic memory allocation functions such as `malloc` and `free`.

• Understand the concept of memory leaks and how to avoid them.

Concepts

Memory management involves allocating and deallocating memory using pointers. Dynamic memory allocation functions such as `malloc` and `free` are used to allocate and deallocate memory. For example, `int *ptr = malloc(sizeof(int));` allocates memory for an integer variable. Memory leaks occur when memory is allocated but not deallocated, leading to memory waste. To avoid memory leaks, it is essential to deallocate memory using `free` when it is no longer needed.

Lab 1: Pointer Practice

Learning Objectives

• Practice using pointers to access and manipulate variables.

• Learn how to use pointer arithmetic to access elements in arrays.

• Understand how to allocate and deallocate memory using pointers.

Instructions

Complete the following exercises to practice using pointers:

• Declare and initialize a pointer to an integer variable.

• Use pointer arithmetic to access elements in an array.

• Allocate and deallocate memory using dynamic memory allocation functions.

Week 12

Week 12: Introduction to Object-Oriented Programming Concepts (3-1)

Lecture Title

Object-Oriented Programming (OOP) Concepts

Learning Objectives

* Understand the basics of Object-Oriented Programming (OOP) concepts

* Learn about encapsulation, inheritance, and polymorphism

* Apply OOP concepts to real-world programming problems

Concepts

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. The core principles of OOP are:

* **Encapsulation**: Bundling data and methods that manipulate that data into a single unit, called a class.

* **Inheritance**: Creating a new class based on an existing class, inheriting its properties and methods.

* **Polymorphism**: The ability of an object to take on multiple forms, depending on the context in which it is used.

OOP provides a way to write reusable, modular, and maintainable code by organizing it into objects and classes. This approach helps to reduce code duplication and improves the overall structure of the program.

Lecture Content

* Introduction to OOP concepts

* Encapsulation, inheritance, and polymorphism

* Real-world examples of OOP in programming

Lab Exercises

* Implement a simple banking system using OOP concepts

* Create a class hierarchy with inheritance and polymorphism

* Write a program that demonstrates encapsulation and data hiding

Note: The lab exercises will be discussed in the lab session, and students will work on implementing the exercises using their preferred programming language.

Week 13

Week 13: Debugging and Error Handling

Lecture 1: Introduction to Debugging Techniques (30 minutes)

Debugging and Error Handling

Learning Objectives:

• Understand the importance of debugging in programming

• Learn basic debugging techniques

Concepts:

• Debugging: the process of identifying and fixing errors in a program

• Types of errors: syntax errors, runtime errors, logical errors

• Basic debugging techniques: print statements, debuggers, logs

Lecture 2: Error Handling in Programming Languages (45 minutes)

Error Handling in Programming Languages

Learning Objectives:

• Understand how programming languages handle errors

• Learn about try-catch blocks and error handling mechanisms

Concepts:

• Error handling in programming languages: exceptions, exceptions handling, error codes

• Try-catch blocks: catching and handling exceptions

• Error handling mechanisms: throw, catch, finally blocks

Lecture 3: Debugging Tools and Techniques (45 minutes)

Debugging Tools and Techniques

Learning Objectives:

• Learn about popular debugging tools and techniques

• Understand how to use debuggers and debug output

Concepts:

• Debugging tools: print statements, debuggers, logs, etc.

• Debugging techniques: stepping through code, inspecting variables, etc.

Lab 1: Debugging Practice (60 minutes)

Lab Exercise: Debugging Practice

Learning Objectives:

• Practice debugging techniques using a programming language of choice

• Apply error handling mechanisms to a programming problem

Concepts:

• Practice debugging using a debugger or print statements

• Apply try-catch blocks and error handling mechanisms to a programming problem

Week 14

Week 14: Introduction to Object-Oriented Programming Concepts (3-0)

Lecture Title

Object-Oriented Programming Fundamentals

Learning Objectives

* Understand the basics of Object-Oriented Programming (OOP)

* Learn about classes, objects, inheritance, polymorphism, and encapsulation

Concepts

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. **Classes** are blueprints or templates that define the properties and behavior of an object. **Objects** are instances of a class and have their own set of attributes (data) and methods (functions).

In OOP, there are four key concepts:

* **Inheritance**: a process where one class inherits properties and behavior from another class.

* **Polymorphism**: the ability of an object to take on multiple forms.

* **Encapsulation**: the idea of bundling data and methods that operate on that data within a single unit.

* **Abstraction**: the practice of exposing only necessary information to the outside world while hiding implementation details.

Understanding OOP concepts is essential for designing and developing complex software systems.

Note: This lecture provides a foundational understanding of OOP concepts. The next lecture will delve deeper into these topics.

Week 15

Week 15: Object-Oriented Programming Concepts (3-1)

Lecture 1: Introduction to Object-Oriented Programming (OOP) Concepts

Learning Objectives:

• Understand the basic principles of Object-Oriented Programming (OOP)

• Learn about the four pillars of OOP: Encapsulation, Abstraction, Inheritance, and Polymorphism

Concepts:

Object-Oriented Programming is a programming paradigm that revolves around the concept of objects and classes. The four pillars of OOP are:

• **Encapsulation**: Hiding the internal details of an object from the outside world and exposing only the necessary information through public methods.

• **Abstraction**: Representing complex systems in a simplified way, focusing on essential features while ignoring non-essential details.

• **Inheritance**: Creating a new class based on an existing class, inheriting its properties and behavior.

• **Polymorphism**: The ability of an object to take on multiple forms, depending on the context in which it is used.

Lecture 2: Classes and Objects

Learning Objectives:

• Understand the concept of classes and objects in OOP

• Learn how to define and create classes and objects

Concepts:

• A **class** is a blueprint or a template that defines the properties and behavior of an object.

• An **object** is an instance of a class, which has its own set of attributes (data) and methods (functions).

• Classes and objects are the fundamental building blocks of OOP.

Lecture 3: Inheritance and Polymorphism

Learning Objectives:

• Understand the concept of inheritance and its types

• Learn about polymorphism and its types

Concepts:

• **Inheritance types**: Single inheritance, Multiple inheritance, Multilevel inheritance.

• **Polymorphism types**: Method overloading, Method overriding, Operator overloading.

Lab 1: Implementing OOP Concepts

Learning Objectives:

• Implement OOP concepts using a programming language

• Practice creating classes, objects, and methods

Instructions:

Implement a simple banking system using OOP concepts. Create a `BankAccount` class with attributes and methods. Demonstrate inheritance and polymorphism by creating a `SavingsAccount` and `CurrentAccount` class that inherit from `BankAccount`.

Week 16

Week 16: Basic Problem Solving and Program Design Techniques (3-1)

Lecture Title: Problem Solving and Program Design Techniques

Learning Objectives

* Understand the importance of problem-solving techniques in programming

* Learn basic program design techniques

* Apply problem-solving and design techniques to real-world programming problems

Concepts

Problem-solving techniques are essential in programming, as they help developers break down complex problems into manageable parts. To design a program, follow these steps:

1. **Problem definition**: Clearly define the problem and its requirements.

2. **Analysis**: Break down the problem into smaller sub-problems.

3. **Design**: Determine the program's architecture and components.

4. **Implementation**: Write the code to implement the design.

5. **Testing**: Test the program to ensure it meets the requirements.

Some basic program design techniques include:

* **Top-down design**: Divide the problem into smaller sub-problems, starting from the overall system.

* **Bottom-up design**: Start with the smallest components and build up to the overall system.

* **Modular design**: Divide the program into independent modules, each with a specific function.

By applying these problem-solving and design techniques, developers can create efficient, maintainable, and scalable programs.

Lab: Implementing Problem-Solving Techniques (1 hour)

* Implement a simple program using top-down design

* Modify the program to use bottom-up design

* Compare the results and discuss the advantages of each approach